

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA SAU ĐẠI HỌC



BÁO CÁO CHUYÊN ĐỀ

**So sánh Bloom Filter và Counting Bloom Filter
trong bài toán lọc dữ liệu tốc độ cao**

Giảng viên: TS. Đỗ Thanh Hà

Học viên thực hiện: NGUYỄN MẠNH CƯỜNG – B25CHHT082
TRẦN DUY KHÁNH – B25CHHT100
NGUYỄN ĐỨC HOÀNG – B25CHHT096

HÀ NỘI – 2026

Mục lục

Danh mục ký hiệu và chữ viết tắt	3
1 Giới thiệu	4
2 Cơ sở lý thuyết	4
2.1 Bloom Filter (Bloom, 1970)	4
2.2 Counting Bloom Filter (Fan <i>et al.</i> , 2000)	4
2.3 Công thức xác suất	5
3 Mã giả	6
4 So sánh định tính	7
5 Ứng dụng trong lọc dữ liệu tốc độ cao	7
5.1 Deep packet inspection và IDS	8
5.2 Lọc truy vấn trong cơ sở dữ liệu LSM-tree	8
5.3 Web cache hợp tác	8
5.4 Khử trùng lặp dữ liệu	8
6 Phương pháp thực nghiệm	8
7 Kết quả và thảo luận	9
7.1 FPR thực nghiệm bám sát lý thuyết (H1, H4)	9
7.2 Số hàm băm tối ưu (H4)	10
7.3 Bộ nhớ: CBF trả giá $4\times$ (H2)	11
7.4 Thông lượng (H3)	11
7.5 Đánh đổi FPR khi cố định payload bộ nhớ	12
7.6 Contract của delete và stress test xóa sai	13
7.7 Đối chiếu với các biến thể (E5)	14
8 Các biến thể liên quan	15
8.1 Blocked Bloom Filter	15
8.2 Cuckoo Filter	16
8.3 d-left Counting Bloom Filter	16
9 Khuyến nghị lựa chọn	17
10 Kết luận	18

Danh sách hình vẽ

1	FPR theo m/n (trục tung log). BF và CBF trùng nhau; thanh sai số là KTC Wilson 95%, đường liền là $p = (1 - e^{-kn/m})^k$ tại $k = k^*$.	10
2	FPR theo số hàm băm k tại $m/n = 10$: cực tiểu tại $k^* = (m/n) \ln 2 \approx 7$.	11
3	Median throughput insert, query (miss) và delete theo m/n ; dải tô là IQR của 5 lượt đo.	12
4	FPR (thang log) khi cả ba mảng backing dùng cùng payload byte.	13
5	FPR (thang log) của năm cấu trúc ở cùng mức ≈ 10 bit/khóa; nhân dưới mỗi cột cho biết cấu trúc có hỗ trợ xóa hay không.	15

6	Blocked Bloom Filter: thay vì rải k bit khắp mảng (trên), toàn bộ k bit của một khóa nằm gọn trong block 512 bit do h_0 chọn (dưới), nên mỗi truy vấn tốn tối đa một cache miss.	15
7	Cuckoo Filter: fingerprint A7 của khóa x có hai bucket ứng viên $i_1 = 2$ và $i_2 = 6$; nó được đặt vào chỗ trống của bucket 2. Khi cả hai bucket đầy, một fingerprint bị đuổi sang bucket thay thế của chính nó (tính bằng XOR), lặp tối đa MAX_KICKS lần.	16
8	d-left CBF với $d = 3$: khóa x băm ra một bucket ứng viên trong mỗi bảng con (viền cam, nhãn $h_i(x)$); mỗi ô vuông nhỏ là một mục đã lưu, nên tải ba ứng viên là 2/1/1 và bucket của bảng con 2 được chọn (viền xanh). Phần phóng to cho thấy mỗi mục gồm một fingerprint kèm counter nhỏ để hỗ trợ đếm bội và xóa. .	17

Danh sách bảng

1	Giá trị lý thuyết: số hàm băm tối ưu và FPR theo tỉ lệ m/n	5
2	So sánh Bloom Filter và Counting Bloom Filter	7
3	E1: FPR lý thuyết vs. thực nghiệm và KTC Wilson 95% ($n = 100\,000$, $k = \text{round}((m/n) \ln 2)$)	10
4	E1: Dung lượng payload của mảng backing theo m/n ($n = 100\,000$)	11
5	E1: Median throughput qua 5 lượt (nghìn thao tác/giây; CBF4 = CBF 4-bit) . .	11
6	E3: Cùng payload 125 000 byte, $n = 100\,000$, k tối ưu cho từng cấu trúc	13
7	E4: Xóa đúng và stress test xóa sai trong CBF	14
8	E5: Năm cấu trúc ở cùng mức ≈ 10 bit/khóa ($n = 100\,000$)	14

Danh mục ký hiệu và chữ viết tắt

Ký hiệu / viết tắt	Ý nghĩa
m	số ô của bộ lọc (bit với BF; counter với CBF)
n	số phần tử đã chèn vào bộ lọc
n'	số khóa vắng mặt dùng để đo FPR
k	số hàm băm; $k^* = (m/n) \ln 2$ là giá trị tối ưu
p	xác suất dương tính giả
b	độ rộng một counter của CBF, tính bằng bit (mặc định $b = 4$)
$C[i]$	giá trị counter thứ i của CBF
λ	kỳ vọng số lượt băm vào một ô, $\lambda = kn/m$
BF	Bloom Filter
CBF	Counting Bloom Filter (4-bit/8-bit chỉ độ rộng counter)
FPR	false positive rate (tỷ lệ dương tính giả)
KTC	khoảng tin cậy (trong báo cáo: khoảng Wilson, mức 95%)
IQR	interquartile range (khoảng tứ phân vị của các lượt đo)
DPI / IDS	deep packet inspection / intrusion detection system
LSM-tree, SSTable	cây log-structured merge; tệp bảng sắp xếp bất biến
payload	phần dữ liệu chính của mảng lưu trữ, không tính overhead
contract của delete	tiền điều kiện: chỉ xóa khóa chắc chắn đã chèn

Tóm tắt nội dung

Báo cáo so sánh Bloom Filter (BF, Bloom 1970) và Counting Bloom Filter (CBF, Fan *et al.* 2000) cho bài toán lọc dữ liệu tốc độ cao (lọc gói tin, deep packet inspection, lọc truy vấn cơ sở dữ liệu, lọc cache, khử trùng lặp). Chúng tôi trình bày cơ sở lý thuyết, công thức xác suất dương tính giả (false positive), phân tích tràn counter, mã giả cho các thao tác insert/query/delete, và một bộ thực nghiệm Python tự cài đặt cả hai cấu trúc. Kết quả trên $n = 100\,000$ khóa xác nhận: (i) tỉ lệ dương tính giả (FPR) thực nghiệm bám sát công thức $p = (1 - e^{-kn/m})^k$ và khi dùng cùng bộ hàm băm, cùng m, k thì BF và CBF cho kết quả truy vấn *trùng nhau từng khóa*; (ii) phần dữ liệu chính (payload) của CBF 4-bit tốn đúng $4 \times$ BF, do đó với cùng ngân sách byte, FPR của CBF (30.1%) cao hơn BF (0.82%) khoảng 37 lần, tức hơn một bậc độ lớn; (iii) CBF hỗ trợ xóa an toàn khi bên gọi (caller) chỉ xóa khóa chắc chắn đã chèn: xóa đúng 20\,000 khóa không làm mất khóa thật nào còn lại, trong khi stress test trên một bộ lọc riêng chứa đủ n khóa, cố ý xóa các false positive, tạo ra 8\,063 false negative (8.06% của n); (iv) ở cùng mức ≈ 10 bit/khóa, trong các cấu trúc hỗ trợ xóa, Cuckoo Filter đạt FPR 2.32%, thấp hơn CBF 4-bit hơn một bậc độ lớn. Kết luận: dùng BF cho tập gần như tĩnh; dùng CBF khi cần xóa động và hệ thống bảo đảm được contract của delete; khi cần xóa với FPR mục tiêu $< 3\%$, Cuckoo Filter là lựa chọn đáng cân nhắc hơn.

1 Giới thiệu

Lọc dữ liệu tốc độ cao quy về bài toán *kiểm tra thành viên tập hợp xấp xỉ* (approximate set-membership): cho tập S và khóa x (gói tin, chữ ký mã độc, khóa cơ sở dữ liệu, fingerprint khối dữ liệu), cần trả lời nhanh “ x chắc chắn không thuộc S ” hoặc “ x có thể thuộc S ” với ngân sách bộ nhớ đủ nhỏ để nằm gọn trong SRAM/cache. Việc chấp nhận một tỉ lệ nhỏ dương tính giả cho phép giảm mạnh không gian lưu trữ so với bảng băm chính xác. Đây là đánh đổi không gian/thời gian kinh điển của Bloom [1].

Các miền ứng dụng tiêu biểu gồm: (i) *lọc gói tin và DPI/IDS*: đối sánh chữ ký Snort bằng Bloom filter song song trên FPGA [8]; (ii) *lọc truy vấn cơ sở dữ liệu* kiểu LSM-tree: RocksDB có thể gắn một BF cho mỗi SSTable để bỏ qua các tệp “chắc chắn không chứa khóa”, tránh I/O đĩa [9]; (iii) *web cache hợp tác*: Cache Digest của Squid và giao thức Summary Cache [2]; (iv) *khử trùng lặp* (deduplication) trên fingerprint khối dữ liệu.

Đóng góp của báo cáo: hệ thống hóa lý thuyết BF/CBF; cài đặt độc lập cả hai cấu trúc (kể cả CBF 4-bit đóng gói nibble đúng nghĩa); và bộ thực nghiệm định lượng bốn khía cạnh: độ chính xác, thông lượng, bộ nhớ, và rủi ro false negative khi xóa.

2 Cơ sở lý thuyết

2.1 Bloom Filter (Bloom, 1970)

BF gồm mảng B có m bit khởi tạo 0 và k hàm băm độc lập, phân bố đều $h_1, \dots, h_k : \mathcal{U} \rightarrow \{0, \dots, m-1\}$. *Insert*(x) đặt $B[h_i(x)] = 1$ với mọi i ; *Query*(x) trả “có thể có” nếu mọi bit tương ứng bằng 1, ngược lại trả “chắc chắn không”. Bit đã bật không bao giờ tự tắt nên BF **không có false negative**, nhưng **có false positive** do va chạm băm. BF **không hỗ trợ xóa**: tắt các bit của x có thể tắt nhầm bit đang được phần tử khác dùng chung.

2.2 Counting Bloom Filter (Fan *et al.*, 2000)

CBF thay mỗi bit bằng một *counter* b bit (thường $b = 4$), ký hiệu $C[i]$. *Insert* tăng, *Delete* giảm k counter tương ứng; *Query* trả “có thể có” khi mọi counter dương. Contract của Delete yêu cầu bên gọi (caller) chỉ xóa khóa chắc chắn đã chèn: câu trả lời “có thể có” của Query không chứng minh membership và không thể dùng làm điều kiện đủ để xóa. Khi contract này được tuân thủ và counter không mất thông tin do tràn, CBF không sinh false negative. Hai nguồn phá vỡ bảo đảm là *tràn counter* khi hơn $2^b - 1$ phần tử băm vào cùng một ô, và *xóa sai* một phần tử chưa từng chèn [2, 3].

2.3 Công thức xác suất

Sau khi chèn n phần tử với k hàm băm, xác suất một bit cụ thể còn 0:

$$p_0 = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}. \quad (1)$$

Xác suất dương tính giả (cả k bit của một khóa lạ đều bằng 1):

$$p \approx \left(1 - e^{-kn/m}\right)^k. \quad (2)$$

Cực tiểu hóa p theo k (tương đương yêu cầu tỉ lệ bit 0 bằng $1/2$, tức entropy cực đại) cho số hàm băm tối ưu và bộ nhớ tối ưu theo FPR mục tiêu p :

$$k^* = \frac{m}{n} \ln 2 \approx 0.693 \frac{m}{n}, \quad m = -\frac{n \ln p}{(\ln 2)^2} \iff \frac{m}{n} \approx 1.44 \log_2 \frac{1}{p}. \quad (3)$$

Tại k^* : $p = 2^{-(m/n) \ln 2} \approx 0.6185^{m/n}$. Quy tắc thực dụng: $m/n = 8$ cho FPR $\approx 2\%$; $m/n = 10$ cho FPR $< 1\%$.

Dẫn xuất công thức (2). Mỗi lần băm chọn một trong m ô với xác suất đều nhau, nên một ô cụ thể *không* bị chọn với xác suất $1 - 1/m$. Chèn n phần tử với k hàm băm tạo ra kn lần băm độc lập, do đó một bit cụ thể vẫn bằng 0 với xác suất $(1 - 1/m)^{kn}$; dùng giới hạn kinh điển $(1 - 1/m)^m \rightarrow e^{-1}$ khi m lớn ta được xấp xỉ $e^{-kn/m}$ trong (1). Một khóa lạ bị báo nhầm “có” khi cả k vị trí của nó đều mang bit 1; xấp xỉ coi k bit này độc lập cho ra (2). Cần lưu ý hai điểm: (i) các bit không hoàn toàn độc lập, nên (2) chỉ là xấp xỉ và trên thực tế là *cận dưới* của FPR thật; độ chệch thấy rõ nhất khi bộ lọc dày đặc (m/n nhỏ), điều mà thực nghiệm ở Mục 7.1 quan sát được trực tiếp; (ii) phân tích giả định hàm băm lý tưởng, phân bố đều.

Dẫn xuất số hàm băm tối ưu. Lấy logarit của (2): $\ln p = k \ln(1 - e^{-kn/m})$. Đặt $t = e^{-kn/m}$ (tỉ lệ bit 0 kỳ vọng), khi đó $k = -(m/n) \ln t$ và $\ln p = -(m/n) \ln t \ln(1-t)$. Biểu thức $\ln t \ln(1-t)$ đối xứng qua $t = 1/2$ và đạt cực đại tại đó, nên p đạt cực tiểu khi $t = 1/2$: nghĩa là bộ lọc tối ưu có *một nửa số bit bằng 0*, tức trạng thái entropy cực đại, khi mỗi bit mang lượng thông tin lớn nhất. Thay $t = 1/2$ vào $k = -(m/n) \ln t$ được $k^* = (m/n) \ln 2$, và $p = 2^{-k^*}$ cho về còn lại của (3). Bảng 1 liệt kê vài giá trị tiêu biểu; sáu dòng đầu chính là các cấu hình được kiểm chứng thực nghiệm ở Mục 7.1.

Bảng 1: Giá trị lý thuyết: số hàm băm tối ưu và FPR theo tỉ lệ m/n

m/n	$k^* = \text{round}((m/n) \ln 2)$	FPR tại k^*
4	3	14.689%
6	4	5.606%
8	6	2.158%
10	7	0.819%
12	8	0.314%
16	11	0.046%
20	14	0.007%

Từ hai hàm băm đến k vị trí. Tính k hàm băm độc lập cho mỗi khóa là tốn kém. Sơ đồ double hashing của Kirsch–Mitzenmacher [5] chỉ cần hai hàm băm h_1, h_2 rồi sinh k vị trí bằng $g_i(x) = (h_1(x) + i h_2(x)) \bmod m$; phân tích chứng minh FPR tiệm cận không đổi so với k hàm độc lập. Một chi tiết cài đặt quan trọng: nếu $\gcd(h_2, m) > 1$ (trường hợp xấu nhất là $h_2 \equiv 0$), dãy g_i suy biến và chỉ phủ một phần mảng; buộc h_2 nhận giá trị lẻ loại trừ rủi ro này với mọi m chẵn (kể cả lũy thừa của 2), tương ứng với phép `h2 |= 1` trong bản cài đặt của chúng tôi (notebook `thuc_nghiem.ipynb`).

Trần counter và vì sao 4 bit là đủ. Số phần tử bấm vào một counter tuân theo phân bố nhị thức $B(nk, 1/m) \approx \text{Poisson}(\lambda)$ với $\lambda = kn/m = \ln 2$ ở cấu hình tối ưu. Do đó

$$\Pr[C[i] = 16] \approx \frac{e^{-\ln 2} (\ln 2)^{16}}{16!} \approx 6.78 \times 10^{-17}, \quad (4)$$

và Fan *et al.* chặn trên toàn cục: với $k \leq (\ln 2)m/n$, $\Pr[\max_i C[i] \geq 16] \leq m \cdot 1.37 \times 10^{-15}$ [2]. Vì xác suất này vô cùng nhỏ (“minuscule”), counter 4 bit (đếm 0..15) đủ cho hầu hết ứng dụng. CBF tốn $4 \times$ payload so với BF cùng số ô [3]. Nếu insert bão hòa ở 15 rồi delete vẫn giảm như counter thường, số lần tăng vượt ngưỡng đã bị mất và false negative có thể xuất hiện sau nhiều lần xóa. Chính sách “sticky saturation” (không bao giờ giảm ô đã bão hòa) tránh lỗi này nhưng đổi lại tạo false positive dai dẳng; do đó hệ thống thực tế cần theo dõi overflow hoặc rebuild từ nguồn dữ liệu chuẩn (source of truth).

Hai hệ quả cho việc so sánh. (1) Với cùng m , k và cùng bộ hàm băm, một ô của CBF dương khi và chỉ khi bit tương ứng của BF bằng 1 (khi chỉ chèn, chưa xóa), nên hai cấu trúc trả lời truy vấn *giống hệt nhau từng khóa*; CBF không “chính xác hơn” BF. (2) Với cùng *ngân sách byte*, BF có số ô gấp b lần CBF b -bit, nên FPR của BF luôn thấp hơn hẳn (Mục 7.5).

3 Mã giả

Thuật toán 1 Bloom Filter: Insert và Query

```

1: procedure BF-Insert( $x$ )
2:   for  $i \leftarrow 1$  to  $k$  do
3:      $B[h_i(x)] \leftarrow 1$ 
4:   end for
5: end procedure
6: procedure BF-Query( $x$ )
7:   for  $i \leftarrow 1$  to  $k$  do
8:     if  $B[h_i(x)] = 0$  then
9:       return False ▷ chắc chắn không
10:    end if
11:  end for
12:  return True ▷ có thể có
13: end procedure

```

Thuật toán 2 Counting Bloom Filter: Insert, Query, Delete (insert bão hòa)

```
1: procedure CBF-Insert( $x$ )
2:   for  $i \leftarrow 1$  to  $k$  do
3:     if  $C[h_i(x)] < 2^b - 1$  then
4:        $C[h_i(x)] \leftarrow C[h_i(x)] + 1$ 
5:     end if
6:   end for
7: end procedure
8: procedure CBF-Query( $x$ )
9:   for  $i \leftarrow 1$  to  $k$  do
10:    if  $C[h_i(x)] = 0$  then
11:      return False
12:    end if
13:  end for
14:  return True
15: end procedure
16: procedure CBF-Delete( $x$ )
17:   Tiền điều kiện: caller biết  $x$  đã được chèn
18:   for  $i \leftarrow 1$  to  $k$  do
19:     if  $C[h_i(x)] > 0$  then
20:        $C[h_i(x)] \leftarrow C[h_i(x)] - 1$ 
21:     end if
22:   end for
23: end procedure
```

▷ bão hòa ở $2^b - 1$

4 So sánh định tính

Bảng 2: So sánh Bloom Filter và Counting Bloom Filter

Tiêu chí	Bloom Filter	Counting Bloom Filter
Đơn vị ô	1 bit	counter b bit ($b=4$)
Bộ nhớ payload	m bit	$b \cdot m$ bit ($\sim 4\times$)
Xóa (delete)	Không	Có, với khóa biết chắc đã chèn
False positive	Có, $p \approx (1 - e^{-kn/m})^k$	Có (cùng công thức)
False negative	Không bao giờ	Không nếu dùng đúng; có thể khi tràn/xóa sai
Insert/Query	$O(k)$	$O(k)$
FPR cùng ngân sách byte	Thấp hơn ($b \times$ số ô)	Cao hơn
Song song hóa cập nhật	Ghi 1 bit an toàn	Đọc-sửa-ghi cần đồng bộ
Hợp phần cứng	Rất tốt (SRAM on-chip)	Tốn SRAM hơn (thường ở phần mềm)
Ứng dụng điển hình	SSTable filter, Cache Digest, chữ ký DPI tĩnh, dedup	Router xóa flow, cache động, Summary Cache cục bộ

5 Ứng dụng trong lọc dữ liệu tốc độ cao

Trước khi vào thực nghiệm, mục này đối chiếu đặc tính BF/CBF với bốn miền ứng dụng tiêu biểu; mỗi miền minh họa một cách phân vai khác nhau giữa cấu trúc gọn nhẹ không hỗ trợ xóa (BF) và cấu trúc hỗ trợ xóa với chi phí bộ nhớ cao hơn (CBF).

5.1 Deep packet inspection và IDS

Dharmapurikar *et al.* [8] đặt các Bloom filter *song song* trong SRAM on-chip của FPGA, mỗi bộ lọc phụ trách một độ dài chữ ký; tại mỗi vị trí byte của luồng dữ liệu, mọi độ dài được kiểm tra đồng thời. Chỉ khi một filter báo “có thể có”, gói tin mới được chuyển sang khối so khớp chính xác; nhờ đó BF đóng vai trò *lọc thô*, loại bỏ phần lớn lưu lượng vô hại ở tốc độ đường truyền. Nguyên mẫu trên Xilinx Virtex 2000E hỗ trợ khoảng 10 000 chữ ký với thông lượng ~ 600 Mbps; các thiết kế phần cứng kế tiếp cùng hướng nhắm tới hàng chục Gbps. Tập chữ ký IDS ít thay đổi nên BF thuần phù hợp; khi cần cập nhật, mô hình lai thường dùng là CBF trong phần mềm điều khiển (nơi giữ nguồn dữ liệu chuẩn, nên delete tuân thủ contract như E4 đòi hỏi) và BF phần cứng được đồng bộ lại từ đó.

5.2 Lọc truy vấn trong cơ sở dữ liệu LSM-tree

Trong RocksDB, mỗi tệp SSTable (bất biến sau khi ghi) mang một Bloom filter riêng; truy vấn điểm kiểm tra filter trước, câu trả lời “chắc chắn không” cho phép bỏ qua tệp mà không tốn I/O đĩa [9], qua đó giảm trực tiếp read amplification. Đáng chú ý, tính bất biến của SSTable khiến nhu cầu xóa khỏi filter *không tồn tại*: compaction tạo tệp mới kèm filter mới, đúng chiến lược “rebuild thay vì delete” mà Mục 9 khuyến nghị. Cấu hình phổ biến khoảng 10 bit/khóa cho FPR quanh 1%, khớp Bảng 1; bản full filter hiện đại còn gói toàn bộ probe vào một cache line (Mục 8).

5.3 Web cache hợp tác

Giao thức Summary Cache [2], nơi CBF ra đời, cho mỗi proxy giữ *tóm tắt* thư mục cache của các proxy láng giềng dưới dạng BF, và chỉ chuyển tiếp yêu cầu khi tóm tắt báo “có thể có”. Vì nội dung cache thay đổi liên tục, mỗi proxy duy trì một CBF *cục bộ* cho thư mục của chính mình (đủ thông tin để xóa an toàn) và định kỳ phát snapshot dạng BF 0/1 cho láng giềng: CBF đứng ở nơi có nguồn dữ liệu chuẩn, BF đứng ở nơi chỉ cần tra cứu. Fan *et al.* báo cáo giảm số thông điệp liên proxy 25–60 lần và băng thông hơn 50% so với giao thức ICP truyền thống, với load factor m/n từ 8 đến 16 [2]. Cache Digest của Squid dùng cùng nguyên lý.

5.4 Khử trùng lặp dữ liệu

Hệ thống sao lưu/khử trùng lặp băm mỗi khối dữ liệu thành fingerprint và giữ chỉ mục đầy đủ trên đĩa; một BF trong RAM trả lời “chắc chắn chưa gặp” giúp ghi khối mới không cần tra chỉ mục [3]. False positive ở đây chỉ gây một lần tra cứu thừa, không ảnh hưởng đáng kể. Khi có thu hồi khối (garbage collection), tập fingerprint co lại; khi đó hoặc dùng CBF, hoặc đơn giản rebuild filter theo chu kỳ thu hồi.

6 Phương pháp thực nghiệm

Môi trường. Python 3.14.4, CPU Intel(R) Core(TM) i7-10700K CPU @ 3.80GHz; các gói mmh3 5.2.1, bitarray 3.10.1, matplotlib 3.11.1, numpy 2.5.2 và pandas 3.0.5. Toàn bộ mã thực nghiệm nằm trong notebook `thuc_nghiem.ipynb`; chạy hết 64 giây.

Cài đặt. Ba cấu trúc được cài đặt từ đầu (notebook `thuc_nghiem.ipynb`): **BF** dùng mảng bit (`bitarray`); **CBF 4-bit** đóng gói 2 counter/byte (nibble packing), đúng chi phí $4\times$ như phân tích lý thuyết; **CBF 8-bit** dùng counter đúng 8 bit (0..255), chi phí $8\times$. Cả ba dùng chung sơ đồ double hashing Kirsch–Mitzenmacher $g_i(x) = (h_1(x) + i h_2(x)) \bmod m$ với h_1, h_2 là MurmurHash3 hai seed khác nhau [5], nhờ đó BF và CBF thăm *cùng các vị trí* với mỗi khóa, qua đó cô lập đúng biến số cần so sánh. Delete chuẩn giả định caller biết khóa đã chèn và không

thêm Query; riêng E4 dùng một hàm có bước kiểm tra Query trước khi xóa (Query guard) để chứng minh guard này vẫn không ngăn được xóa sai khi gặp false positive.

Dữ liệu và tham số. $n = 100\,000$ khóa “có mặt” (P0000000000...) được chèn và $n' = 100\,000$ khóa “vắng mặt” (A0000000000...) dùng đo FPR; sinh tất định để tái lập được. Tỷ lệ $m/n \in \{4, 6, 8, 10, 12, 16\}$; $k = \text{round}((m/n) \ln 2)$ trừ khi quét k . FPR được báo cùng khoảng tin cậy (KTC) Wilson 95%. Các phép đo throughput lặp 5 lượt, dùng median làm giá trị trung tâm và IQR làm độ phân tán; insert/delete dùng filter mới ở mỗi lượt để bảo đảm trạng thái đầu vào giống nhau.

Năm thực nghiệm.

- E1** FPR thực nghiệm vs. lý thuyết (kèm KTC 95%) và median/IQR throughput của các thao tác insert, query, delete theo m/n .
- E2** FPR theo $k \in \{1, \dots, 12\}$ tại $m/n = 10$ để kiểm chứng $k^* = (m/n) \ln 2$.
- E3** Cố định payload 125 000 byte, so đánh đổi FPR của BF (1 bit/ô), CBF 4-bit và CBF 8-bit.
- E4** Kiểm tra xóa đúng không làm mất khóa thật còn lại, sau đó stress test việc xóa khóa vắng mặt bị CBF báo nhầm “có”.
- E5** Đối chiếu với hai biến thể ở cùng mức ≈ 10 bit/khóa: Blocked Bloom Filter (đánh đổi FPR lấy tính cục bộ cache) và Cuckoo Filter (fingerprint 8 bit, hỗ trợ xóa).

Giả thuyết. **H1:** FPR thực nghiệm của BF và CBF (cùng m, k) trùng nhau và gần đường lý thuyết trong sai số lấy mẫu. **H2:** payload CBF 4-bit = $4 \times$ BF (8-bit = $8 \times$). **H3:** trong cài đặt Python đang xét, throughput BF $\geq$ CBF 4-bit cho các thao tác chung (CBF 8-bit có thể bám sát BF ở query trong biên độ nhiễu). **H4:** FPR giảm mũ theo m/n ; cực tiểu theo k đạt quanh k^* .

7 Kết quả và thảo luận

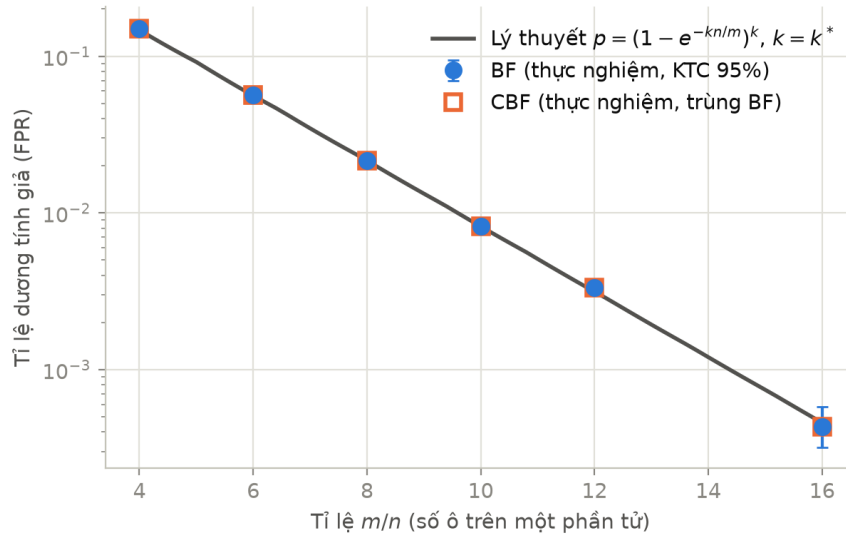
7.1 FPR thực nghiệm bám sát lý thuyết (H1, H4)

Bảng 3 và Hình 1 cho thấy FPR thực nghiệm bám sát công thức (2) trên toàn dải m/n : tại $m/n = 8$ lý thuyết 2.16% so với đo được 2.16%; tại $m/n = 10$ lý thuyết 0.82% so với 0.82%, với KTC Wilson 95% [0.765%, 0.877%]. Tỷ lệ bit 1 đo được tại $m/n = 10$ là 0.503 (lý thuyết 0.503, xấp xỉ mức $1/2$ entropy cực đại). Đúng như phân tích ở Mục 2.3, cột FPR của BF và CBF *trùng nhau từng khóa* (được kiểm tra bằng `assert` cho cả CBF 4-bit và 8-bit): khi chỉ chèn, CBF chính là BF nhìn qua phép thử “counter dương”. Khoảng tin cậy chỉ phản ánh sai số trên tập truy vấn vắng mặt cố định; thí nghiệm chưa lấy trung bình qua nhiều họ hash/seed.

Cột KTC còn cho phép một kiểm định tinh hơn: ở 5/6 cấu hình, giá trị lý thuyết nằm *trong* KTC của giá trị đo. Ngoại lệ duy nhất là $m/n = 4$: lý thuyết 14.689% nằm ngay dưới cận dưới 14.713% của KTC. Đây không phải mâu thuẫn mà là bằng chứng thực nghiệm cho tính chất đã nêu ở Mục 2.3: công thức (2) là xấp xỉ *cận dưới* của FPR thật, và độ chênh lệch rõ nhất khi bộ lọc dày đặc (ở $m/n = 4$ tỷ lệ bit 1 lên tới 0.529). Khi bộ lọc thưa dần, độ chênh giảm nhanh và hai giá trị trùng khớp tới ba chữ số có nghĩa.

Bảng 3: E1: FPR lý thuyết vs. thực nghiệm và KTC Wilson 95% ($n = 100\,000$, $k = \text{round}((m/n) \ln 2)$)

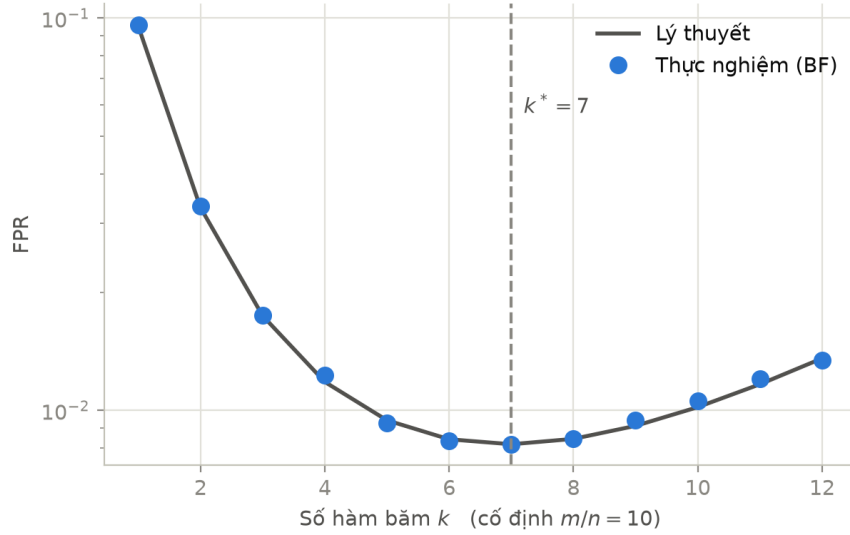
m/n	k	FPR lý thuyết	FPR đo (BF $\equiv$ CBF)	KTC 95%	Tỉ lệ bit 1 (đo)	(lý thuyết)
4	3	14.7%	14.9%	[14.7%, 15.2%]	0.529	0.528
6	4	5.606%	5.650%	[5.509%, 5.795%]	0.487	0.487
8	6	2.158%	2.164%	[2.076%, 2.256%]	0.528	0.528
10	7	0.819%	0.819%	[0.765%, 0.877%]	0.503	0.503
12	8	0.314%	0.334%	[0.300%, 0.372%]	0.487	0.487
16	11	0.046%	0.043%	[0.032%, 0.058%]	0.497	0.497



Hình 1: FPR theo m/n (trục tung log). BF và CBF trùng nhau; thanh sai số là KTC Wilson 95%, đường liền là $p = (1 - e^{-kn/m})^k$ tại $k = k^*$.

7.2 Số hàm băm tối ưu (H4)

Hình 2 quét k từ 1 đến 12 tại $m/n = 10$: FPR đạt cực tiểu quanh $k^* = 7$ đúng như công thức (3). Cơ chế: k quá nhỏ cho một khóa lạ quá ít phép thử độc lập nên hiếm cơ hội bắt gặp được một bit 0; k quá lớn làm đầy mảng bit khiến từng phép thử dễ trùng bit 1 hơn.



Hình 2: FPR theo số hàm băm k tại $m/n = 10$: cực tiểu tại $k^* = (m/n) \ln 2 \approx 7$.

7.3 Bộ nhớ: CBF trả giá $4\times$ (H2)

Bảng 4 xác nhận dung lượng payload của mảng backing: CBF 4-bit (nibble packing) tốn đúng $4.0\times$ BF ở mọi cấu hình; CBF 8-bit tốn $8\times$. Tại $m/n = 10$: BF dùng 125 000 byte, CBF 4-bit 500 000 byte, CBF 8-bit 1 000 000 byte. Các số này không bao gồm object header, allocator hay RSS của tiến trình; hệ số $4\times/8\times$ được suy ra trực tiếp từ cách biểu diễn, không phải một phát hiện về runtime.

Bảng 4: E1: Dung lượng payload của mảng backing theo m/n ($n = 100\,000$)

m/n	m (ô)	BF (byte)	CBF 4-bit (byte)	CBF 8-bit (byte)	CBF4/BF
4	400 000	50 000	200 000	400 000	$4.0\times$
6	600 000	75 000	300 000	600 000	$4.0\times$
8	800 000	100 000	400 000	800 000	$4.0\times$
10	1 000 000	125 000	500 000	1 000 000	$4.0\times$
12	1 200 000	150 000	600 000	1 200 000	$4.0\times$
16	1 600 000	200 000	800 000	1 600 000	$4.0\times$

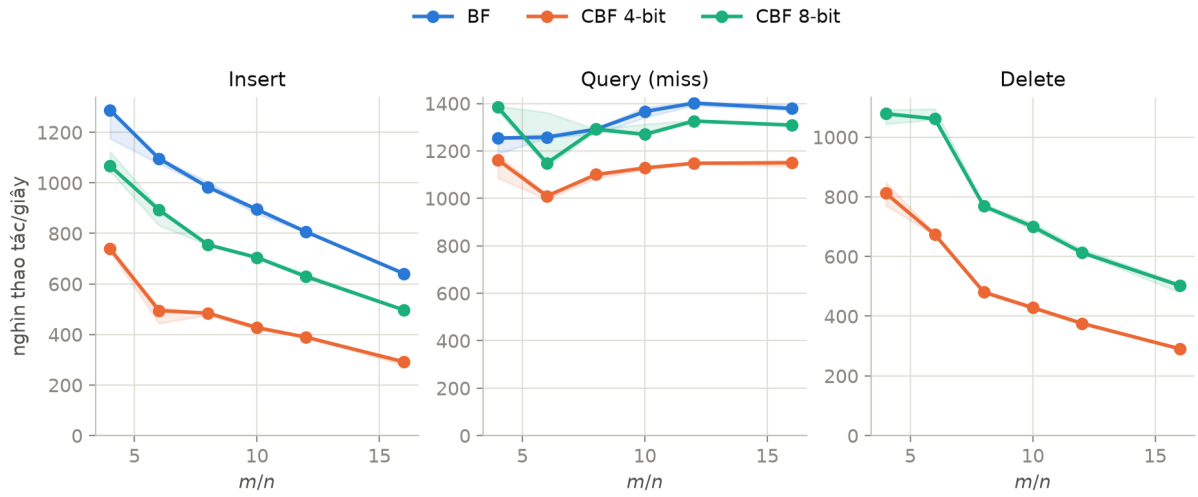
7.4 Thông lượng (H3)

Bảng 5: E1: Median throughput qua 5 lượt (nghìn thao tác/giây; CBF4 = CBF 4-bit)

m/n	k	Insert		Query (miss)		Query (hit) / Delete	
		BF	CBF4	BF	CBF4	BF hit	CBF4 del
4	3	1 288	739	1 254	1 162	1 150	812
6	4	1 096	495	1 258	1 009	987	673
8	6	983	484	1 291	1 100	772	480
10	7	895	427	1 366	1 128	726	428
12	8	806	389	1 401	1 148	674	376
16	11	641	292	1 379	1 150	556	291

Tại $m/n = 10$ (Bảng 5, Hình 3): BF đạt 895 nghìn insert/s so với 427 của CBF 4-bit (nhẹ gấp 2.09 lần) vì CBF phải đọc-sửa-ghi nibble thay vì chỉ bật bit. Với query trên khóa vắng mặt, BF đạt 1 366 nghìn thao tác/s so với 1 128 của CBF (gấp 1.21 lần). Query trên khóa vắng mặt (miss) nhanh hơn trên khóa có mặt (hit): với BF là 1 366 so với 726 nghìn thao tác/s, vì truy vấn miss thoát sớm ngay khi gặp bit 0 đầu tiên, còn truy vấn hit luôn phải thăm đủ k vị trí. Delete chuẩn của CBF 4-bit đạt 428 nghìn thao tác/s; phép đo giả định caller biết khóa đã chèn và không cộng thêm một lượt Query. Mỗi điểm là median của 5 lượt, dải tô trên Hình 3 là IQR. Các số và tỉ số phản ánh bản cài đặt cụ thể dùng `bitarray/bytearray`, thao tác nibble và vòng lặp Python; chúng không chứng minh mọi cài đặt BF luôn nhanh hơn mọi cài đặt CBF.

Hai xu hướng trong Bảng 5 đáng bình luận thêm. Thứ nhất, throughput của *mọi* thao tác giảm dần theo m/n : nguyên nhân không phải kích thước mảng mà là $k = \text{round}((m/n) \ln 2)$ tăng theo, khiến mỗi thao tác phải bấm và thăm nhiều vị trí hơn; đây là chi phí ít được đề cập của việc hạ FPR bằng cách tăng m/n . Thứ hai, trong nội bộ CBF, bản 8-bit nhanh hơn bản 4-bit ở mọi thao tác vì không phải tách/ghép nibble, nhưng tốn gấp đôi bộ nhớ: ngay trong một họ cấu trúc cũng tồn tại đánh đổi tốc độ-dung lượng, và lựa chọn 4-bit của Fan *et al.* là điểm cân bằng nghiêng về bộ nhớ.



Hình 3: Median throughput insert, query (miss) và delete theo m/n ; dải tô là IQR của 5 lượt đo.

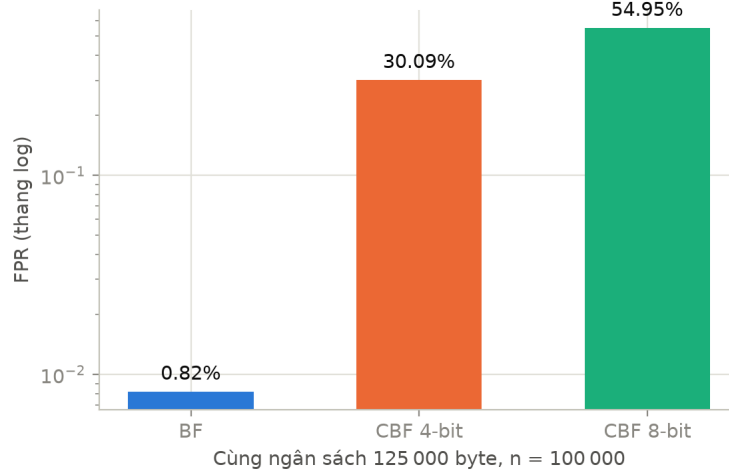
7.5 Đánh đổi FPR khi cố định payload bộ nhớ

Khi dung lượng payload là ràng buộc cứng (ví dụ SRAM on-chip), phép so này cho thấy chi phí FPR của counter; nó không làm hai cấu trúc tương đương chức năng vì BF không hỗ trợ delete. Với cùng 125 000 byte (Bảng 6, Hình 4): BF có 1 000 000 ô nên FPR chỉ 0.82%; CBF 4-bit còn 250 000 ô, FPR vọt lên 30.1%; CBF 8-bit còn 125 000 ô, FPR 54.9%. Khoảng cách 37–67 lần (từ hơn một tới gần hai bậc độ lớn) là đánh đổi dung lượng–FPR để có counter và khả năng xóa.

Kết quả đo cũng được lý thuyết tiên đoán trọn vẹn: với 250 000 ô và $k = 2$, công thức (2) cho 30.3% trong khi giá trị đo là 30.1%; với 125 000 ô và $k = 1$: lý thuyết 55.1%, đo 54.9% (Bảng 6). Nói cách khác, chi phí FPR của counter có thể được tiên lượng đầy đủ ngay từ khâu thiết kế mà không cần đo đạc.

Bảng 6: E3: Cùng payload 125 000 byte, $n = 100\,000$, k tối ưu cho từng cấu trúc

Cấu trúc	Số ô	bit/ô	k	FPR lý thuyết	FPR thực nghiệm
BF	1 000 000	1	7	0.819%	0.819%
CBF 4-bit	250 000	4	2	30.3%	30.1%
CBF 8-bit	125 000	8	1	55.1%	54.9%



Hình 4: FPR (thang log) khi cả ba mảng backing dùng cùng payload byte.

7.6 Contract của delete và stress test xóa sai

E4 (Bảng 7) trước hết kiểm tra trường hợp dùng đúng: tại $m/n = 8$, counter lớn nhất sau insert chỉ là 8 nên không có overflow; sau khi xóa đúng 20 000 khóa, cả 80 000 khóa thật còn lại vẫn được Query nhận ra (0 false negative).

Phần hai dùng một bộ lọc riêng đã chèn đủ $n = 100\,000$ khóa (chưa xóa khóa nào) và cố ý vi phạm contract của delete: hệ thống nhằm câu trả lời “có thể có” là bằng chứng membership và thử xóa 2 164 khóa chưa từng được chèn. Dù dùng Query guard, 2 041 yêu cầu vẫn lọt qua vì chính các khóa này là false positive. Các lần giảm nhằm counter làm 8 063 phần tử thật (8.06% của n) biến mất khỏi bộ lọc. Đây là kết quả có điều kiện của stress test với hàng nghìn yêu cầu xóa sai, *không phải* tỉ lệ false negative mặc định của CBF và không được tổng quát hóa sang workload dùng đúng. Biện pháp chính là chỉ xóa khóa được xác nhận từ nguồn dữ liệu chuẩn; rebuild giúp khôi phục nếu contract hoặc thông tin counter đã bị phá vỡ.

Về cơ chế, quy mô thiệt hại là ước lượng được: 2 041 lần xóa lọt qua guard, mỗi lần giảm $k = 6$ counter, tổng cộng 12 246 lần giảm sai. Ở cấu hình $m/n = 8$, trung bình mỗi ô gánh $\lambda = kn/m = 0.75$ lượt bấm của khóa thật, nên mỗi lần giảm sai có xác suất đáng kể đưa một counter đang phục vụ khóa thật về 0; số lần giảm ở quy mô này vì vậy ảnh hưởng tới nhiều nghìn khóa, và con số 8 063 (8.06%) phù hợp với bậc độ lớn kỳ vọng. Rủi ro vận hành không nằm ở thiệt hại tức thời mà ở tính *tích lũy*: mỗi chu kỳ xóa sai tiếp tục làm suy giảm bộ lọc, và không có cơ chế tự phục hồi nào ngoài việc rebuild.

Bảng 7: E4: Xóa đúng và stress test xóa sai trong CBF

Dại lượng	Giá trị
Cấu hình	$m/n = 8, k = 6, n = 100\,000$
Counter lớn nhất sau insert	8
Xóa đúng / FN trong khóa thật còn lại	20 000 / 0
Khóa vắng mặt bị báo nhầm “có” (false positive)	2 164
Lần xóa sai lọt qua query guard	2 041
Phần tử thật bị mất trong stress test	8 063
Tỉ lệ false negative trong stress test	8.063%

7.7 Đối chiếu với các biến thể (E5)

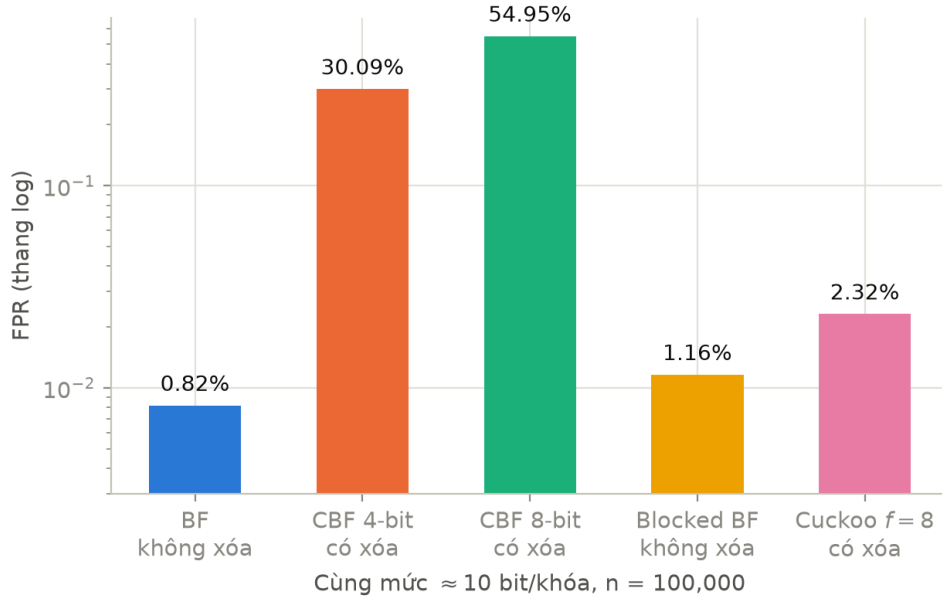
Để đặt BF/CBF cạnh các thiết kế hiện đại hơn, E5 cài đặt thêm Blocked Bloom Filter và Cuckoo Filter rồi so sánh ở cùng mức ≈ 10 bit/khóa (Bảng 8, Hình 5); cơ chế của từng biến thể được trình bày ở Mục 8.

Blocked BF với block 512 bit đạt FPR 1.16% so với 0.82% của BF chuẩn cùng m, k : phần thuế FPR trả cho tính cục bộ cache là có thật nhưng nhỏ. Trong cài đặt phần cứng hoặc C/C++, một cache miss mỗi truy vấn thay vì tối đa k thường bù lại xứng đáng; lợi ích này nằm ngoài phạm vi đo của Python.

Cuckoo Filter (fingerprint $f = 8$ bit, bucket 4 chỗ, 2^{15} bucket) chèn đủ 100 000 khóa không thất bại lần nào (hệ số tải 76.3%), đạt FPR 2.32% với payload 131 072 byte (10.49 bit/khóa), và xóa toàn bộ khóa thành công (0 khóa sót lại). Điểm đáng chú ý nhất khi đặt cạnh nhau: trong các cấu trúc *hỗ trợ xóa*, Cuckoo Filter cho FPR thấp hơn CBF 4-bit hơn một bậc độ lớn (2.32% so với 30.1%); điều này củng cố bằng số đo cho khuyến nghị ưu tiên Cuckoo Filter khi cần xóa với yêu cầu FPR chặt. Ba lưu ý về phạm vi: payload của Cuckoo là danh nghĩa (f bit mỗi chỗ); hệ số tải mới đạt 76.3%, chưa ép tới ngưỡng $\sim 95\%$; và bản cài đặt chưa dùng semi-sorting nên chưa phản ánh cấu hình tối ưu không gian của công trình gốc [7].

Bảng 8: E5: Năm cấu trúc ở cùng mức ≈ 10 bit/khóa ($n = 100\,000$)

Cấu trúc	Payload (byte)	bit/khóa	Hỗ trợ xóa	FPR đo
BF	125 000	10.00	Không	0.819%
Blocked BF	125 056	10.00	Không	1.163%
CBF 4-bit	125 000	10.00	Có	30.1%
CBF 8-bit	125 000	10.00	Có	54.9%
Cuckoo Filter ($f = 8$)	131 072	10.49	Có	2.320%



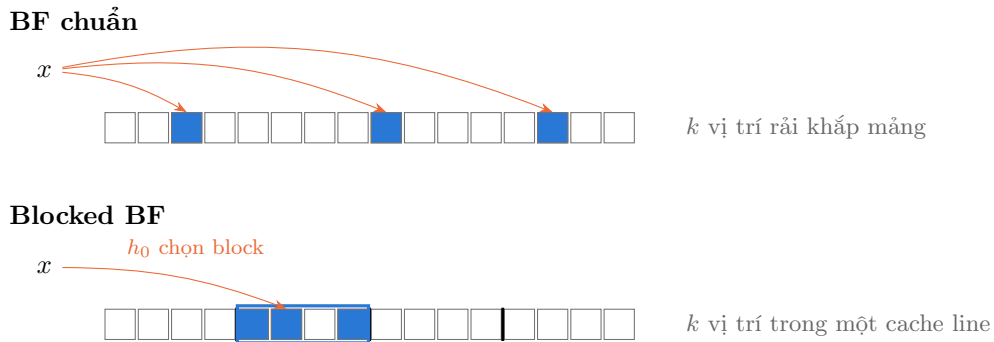
Hình 5: FPR (thang log) của năm cấu trúc ở cùng mức ≈ 10 bit/khóa; nhãn dưới mỗi cột cho biết cấu trúc có hỗ trợ xóa hay không.

8 Các biến thể liên quan

Ba biến thể dưới đây lần lượt khắc phục các hạn chế khác nhau của cặp BF/CBF: chi phí cache miss, chi phí bộ nhớ của khả năng xóa, và cả hai yếu tố cùng lúc. Cơ chế của từng biến thể được minh họa trong các Hình 6–8; hai biến thể đầu đã được đối chiếu bằng số đo trong Mục 7.7.

8.1 Blocked Bloom Filter

Putze *et al.* [4] nhận xét rằng với bộ lọc lớn hơn cache, k lần thăm bit là k lần truy cập bộ nhớ ngẫu nhiên, nghĩa là tối đa k cache miss mỗi truy vấn. Blocked Bloom Filter chia mảng thành các block bằng đúng một cache line (thường $B = 512$ bit): một hàm băm phụ h_0 chọn block, rồi toàn bộ k bit của khóa được đặt *bên trong* block đó (Hình 6). Mỗi truy vấn vì vậy chỉ chạm đúng một cache line: số cache miss giảm từ tối đa k xuống ~ 1 , và mọi phép thử sau lần nạp đầu tiên đều diễn ra trên thanh ghi/cache L1.



Hình 6: Blocked Bloom Filter: thay vì rải k bit khắp mảng (trên), toàn bộ k bit của một khóa nằm gọn trong block 512 bit do h_0 chọn (dưới), nên mỗi truy vấn tốn tối đa một cache miss.

Chi phí đánh đổi nằm ở sự mất cân bằng tải: số khóa rơi vào một block xấp xỉ Poisson với kỳ vọng nB/m , nên luôn có block nhận nhiều khóa hơn mức trung bình và trở nên dày đặc

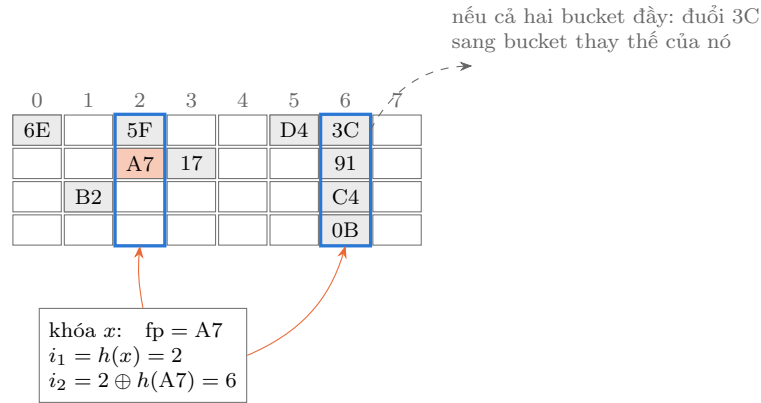
hơn. Vì FPR là hàm lồi của tải, trung bình FPR trên các block cao hơn FPR của bộ lọc cân bằng hoàn hảo (bất đẳng thức Jensen); đây chính là phần chênh 1.16% so với 0.82% mà E5 đo được (Mục 7.7); trong thực tế người ta bù bằng cách cấp thêm một lượng nhỏ bit cho mỗi khóa. RocksDB dùng đúng chiến lược cache-line-local này trong full filter [9]. Như BF gốc, cấu trúc này không hỗ trợ xóa.

8.2 Cuckoo Filter

Cuckoo Filter [7] khác biệt căn bản với mô hình mảng bit: mỗi khóa được đại diện bằng một *fingerprint* f bit lưu tường minh trong bảng băm cuckoo gồm các bucket b chỗ (thường $b = 4$). Mỗi khóa có đúng hai bucket ứng viên, xác định bằng sơ đồ *partial-key cuckoo hashing* (Hình 7):

$$i_1 = h(x), \quad i_2 = i_1 \oplus h(\text{fp}(x)),$$

trong đó phép XOR có tính đối hợp: từ bucket đang chứa fingerprint bất kỳ luôn tính được bucket thay thế mà *không cần khóa gốc*; đây là điều kiện thiết yếu để dời chỗ (eviction) hoạt động khi bảng chỉ lưu fingerprint.



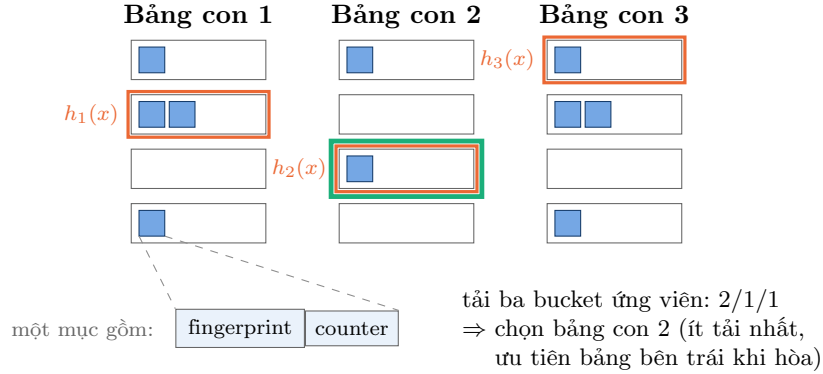
Hình 7: Cuckoo Filter: fingerprint A7 của khóa x có hai bucket ứng viên $i_1 = 2$ và $i_2 = 6$; nó được đặt vào chỗ trống của bucket 2. Khi cả hai bucket đầy, một fingerprint bị đuổi sang bucket thay thế của chính nó (tính bằng XOR), lặp tối đa MAX_KICKS lần.

Truy vấn chỉ đọc hai bucket và so tối đa $2b$ fingerprint; mỗi phép so trùng ngẫu nhiên với xác suất $\sim 2^{-f}$, nên $\varepsilon \approx 2b\alpha/2^f$ với α là hệ số tải. Ở cấu hình E5 ($f = 8$, $b = 4$, $\alpha = 76.3\%$) công thức cho $\approx 2.4\%$, khớp số đo 2.32% (Mục 7.7). Nhờ lưu tường minh từng phần tử, *xóa được* bằng cách gỡ một bản sao fingerprint khỏi một trong hai bucket; đáng chú ý, delete ở đây mang *cùng contract* với CBF: chỉ được xóa khóa chắc chắn đã chen, vì gỡ fingerprint của khóa chưa chen có thể gỡ nhầm fingerprint trùng của khóa khác và gây false negative [7]. Về không gian, chi phí là f/α bit/khóa; với kỹ thuật semi-sorting (tiết kiệm 1 bit mỗi fingerprint), khi FPR mục tiêu $\varepsilon < 3\%$ Cuckoo Filter cần ít bit hơn cả BF tối ưu không gian, nghĩa là vừa hỗ trợ xóa vừa tiết kiệm không gian hơn. Nhược điểm: bucket $b = 4$ cho phép đẩy hệ số tải tới $\sim 95\%$, nhưng vượt ngưỡng đó thao tác chen có thể thất bại (chuỗi dời chỗ quá dài), nên hệ thống phải theo dõi tải và mở rộng bảng kịp thời.

8.3 d-left Counting Bloom Filter

dlCBF [6] thay mảng counter thừa của CBF bằng bảng băm d-left: bảng chia thành d bảng con, mỗi phần tử băm ra một bucket ứng viên trong *mỗi* bảng con và được đặt vào bucket ít tải nhất, ưu tiên bảng bên trái khi hòa (Hình 8). Quy tắc “chọn nơi ít tải trong d lựa chọn” làm tải cực đại tập trung mạnh quanh trung bình, nên các bucket có thể định cỡ cố định mà gần như

không lãng phí chỗ; đây là một tính chất quý cho phần cứng vì d bảng con truy cập được song song với độ trễ cố định.



Hình 8: d-left CBF với $d = 3$: khóa x bấm ra một bucket ứng viên trong mỗi bảng con (viền cam, nhãn $h_i(x)$); mỗi ô vuông nhỏ là một mục đã lưu, nên tải ba ứng viên là 2/1/1 và bucket của bảng con 2 được chọn (viền xanh). Phần phóng to cho thấy mỗi mục gồm một fingerprint kèm counter nhỏ để hỗ trợ đếm bội và xóa.

Mỗi ô của bucket lưu một fingerprint kèm counter nhỏ: *insert* tìm fingerprint trong các bucket ứng viên để tăng counter (phần tử trùng) hoặc chiếm một ô trống; *query* so fingerprint trên d bucket ứng viên; *delete* giảm counter tương ứng. False positive chỉ phát sinh khi một khóa lạ trùng cả bucket ứng viên lẫn fingerprint với một phần tử đã lưu. Vì chỉ những phần tử thực sự có mặt mới chiếm chỗ (thay vì cấp counter cho mọi ô kể cả ô trống như CBF), dlCBF đạt cùng chức năng đếm/xóa với khoảng *một nửa* không gian của CBF 4-bit [6]. Cái giá là logic phức tạp hơn đáng kể: bài gốc phải dùng thêm các phép hoán vị để giữ insert/delete nhất quán khi cùng một phần tử có thể được nhìn thấy ở các bảng con khác nhau. Đây là lý do báo cáo giữ dlCBF ở mức phân tích lý thuyết thay vì cài đặt trong E5.

9 Khuyến nghị lựa chọn

1. Tập tính hoặc rebuild được:

(SSTable bất biến, Cache Digest, chữ ký DPI ít đổi) dùng **BF**.

Chọn $m = -n \ln p / (\ln 2)^2$ và $k = \text{round}((m/n) \ln 2)$ theo FPR mục tiêu p (ví dụ $p = 1\% \Rightarrow m/n \approx 9.6, k = 7$).

2. Cần xóa động hoặc đếm bội:

dùng **CBF 4-bit** khi hệ thống có nguồn dữ liệu chuẩn để bảo đảm chỉ xóa khóa đã chèn. Theo dõi overflow; rebuild từ nguồn dữ liệu chuẩn khi counter mất thông tin hoặc contract delete có thể đã bị vi phạm.

3. Cần xóa và bộ nhớ/FPR là ràng buộc chặt: cân nhắc **Cuckoo Filter**, đặc biệt khi FPR mục tiêu $< 3\%$, hoặc **dlCBF** trên phần cứng; theo dõi hệ số tải và thất bại insert.

4. Nghẽn vì cache miss: chuyển sang **Blocked Bloom Filter**, chấp nhận FPR tăng nhẹ.

Ví dụ thiết kế tham số. Yêu cầu: lọc $n = 10^7$ khóa với FPR mục tiêu $p = 1\%$, tập gần như tĩnh. Áp dụng (3):

$$m = -\frac{n \ln(0.01)}{(\ln 2)^2} \approx 9.59 \times 10^7 \text{ bit} \approx 11.4 \text{ MiB}, \quad \frac{m}{n} \approx 9.59, \quad k = \text{round}(9.59 \ln 2) = 7.$$

Thay lại (2) để kiểm tra: $FPR = 1.00\%$, đạt mục tiêu. Nếu cùng số ô phải chuyển sang counter để xóa được: CBF 4-bit tốn 45.7 MiB, CBF 8-bit tốn 91.4 MiB, tức chênh lệch hơn 34 MiB so với BF chỉ để đổi lấy thao tác delete. Do tập gần như tĩnh, lựa chọn phù hợp là BF kèm lịch rebuild; chỉ khi xuất hiện yêu cầu xóa tức thời mới cân nhắc chi phí của CBF, hoặc chuyển sang Cuckoo Filter nếu FPR mục tiêu vẫn ở mức chặt (Mục 8).

10 Kết luận

Thực nghiệm xác nhận bốn giả thuyết: FPR của BF/CBF trùng nhau và bám công thức lý thuyết trong sai số lấy mẫu (H1), payload CBF 4-bit bằng $4 \times$ BF (H2), và trong cài đặt Python đang xét BF có median throughput cao hơn CBF 4-bit ở các thao tác chung (H3); FPR giảm mũ theo m/n với cực tiểu theo k tại k^* (H4). Bức tranh tổng thể: *CBF đạt được khả năng xóa bằng cách đánh đổi payload counter lớn hơn và một contract delete chặt chẽ hơn*. Khi chỉ xóa khóa chắc chắn đã chèn và không overflow, thí nghiệm không quan sát false negative; false negative xuất hiện trong stress test khi cố ý xóa các khóa vắng mặt. Do đó BF phù hợp với tập tĩnh/rebuild được; CBF phù hợp với tập động khi có nguồn dữ liệu chuẩn cho delete; và Cuckoo Filter, với FPR thấp hơn CBF 4-bit hơn một bậc độ lớn ở cùng ngân sách trong E5, đáng được ưu tiên khảo sát khi cần xóa với yêu cầu FPR/bộ nhớ chặt.

Hạn chế. Benchmark dùng một bản cài đặt Python cụ thể; dù đã báo median/IQR qua 5 lượt, cả số tuyệt đối lẫn tỉ số không đại diện cho mọi bản cài đặt C/C++/phần cứng. FPR dùng một tập khóa tắt định và một cặp seed; KTC Wilson chỉ phản ánh sai số trên các truy vấn vắng mặt, chưa bao phủ biến thiên qua nhiều họ hash/seed. Công thức (2) là xấp xỉ (và là cận dưới); với m nhỏ nên đo thực nghiệm. Phân tích giả định hàm băm lý tưởng; double hashing chỉ hợp lệ tiệm cận và môi trường đối kháng cần hàm băm có khóa. Các số bộ nhớ chỉ là payload của mảng backing, không phải RSS. Bản cài đặt Blocked BF và Cuckoo Filter trong E5 chỉ nhằm đối chiếu FPR và chức năng xóa; chúng chưa tối ưu không gian (semi-sorting) lẫn tốc độ, nên không dùng để so sánh throughput.

Tài liệu

- [1] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [2] L. Fan, P. Cao, J. Almeida, and A. Z. Broder, “Summary Cache: A Scalable Wide-Area Web Cache Sharing Protocol,” *IEEE/ACM Transactions on Networking*, vol. 8, no. 3, pp. 281–293, 2000.
- [3] A. Broder and M. Mitzenmacher, “Network Applications of Bloom Filters: A Survey,” *Internet Mathematics*, vol. 1, no. 4, pp. 485–509, 2004.
- [4] F. Putze, P. Sanders, and J. Singler, “Cache-, hash-, and space-efficient Bloom filters,” *ACM Journal of Experimental Algorithmics*, vol. 14, 2009.
- [5] A. Kirsch and M. Mitzenmacher, “Less hashing, same performance: Building a better Bloom filter,” *Random Structures & Algorithms*, vol. 33, no. 2, pp. 187–218, 2008.
- [6] F. Bonomi, M. Mitzenmacher, R. Panigrahy, S. Singh, and G. Varghese, “An Improved Construction for Counting Bloom Filters,” in *Algorithms: ESA 2006*, LNCS 4168, pp. 684–695.
- [7] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, “Cuckoo Filter: Practically Better Than Bloom,” in *Proc. 10th ACM CoNEXT*, pp. 75–88, 2014.

- [8] S. Dharmapurikar, P. Krishnamurthy, T. Sproull, and J. Lockwood, “Deep Packet Inspection using Parallel Bloom Filters,” in *IEEE Hot Interconnects 11*, 2003.
- [9] RocksDB, “RocksDB Bloom Filter,” project documentation, accessed Aug. 10, 2026.